The Journal of Open Source Software

NeuRodent: a Python Package and Pipeline for Unifying Rodent EEG Analyses

Joseph P. Dong [1], **Yongtaek Oh** [1], **Yastika Singh** [1,2], **Yifan Wei** [1,2], **and Eric D. Marsh** [1,2]

1 Children's Hospital of Philadelphia, United States of America **2** University of Pennsylvania, United States of America

DOI: 10.21105/joss.11032

Software

- Review
- Repository
- Archive

Editor: Charlotte Soneson

Reviewers:

- @JuliusWelzel
- @AmeerHamoodi

Submitted: 02 April 2026
Published: 16 October 2026

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Summary

NeuRodent is a Python package for the quantitative analysis of rodent electroencephalography (EEG) and local field potential (LFP) recordings, which measure voltage from electrodes placed on or in the brain. It loads EEG/LFP recordings and computes features across recording channels and time. Features include signal amplitude, power in standard brainwave frequency bands, power spectrum slope, coherence/correlation between pairs of channels, and voltage spikes that occur when many neurons are activated at once. Results are saved as tables and can be visualized for single animal or multiple animal recordings, enabling comparison across experimental groups.

Small analyses can be run from a script or a Jupyter notebook. Larger studies may use the accompanying Snakemake pipeline, which speeds up the same analyses for a full set of animal recordings by using a computing cluster. NeuRodent is intended for rodent EEG researchers who need reproducible and standardized signal analyses across their studies.

Statement of Need

Electroencephalography (EEG) and its invasive counterpart, local field potential (LFP) recording, are cornerstone neuroscience techniques to measure cortical brain activity across species from humans to rodents. EEG has had over a century of use in clinical settings (Tudor et al., 2005) and is widely used today to diagnose and localize epilepsy, as well as in animal models to validate and develop mechanistic understandings of neurological diseases (Marshall et al., 2021). Historically, EEG interpretation has been qualitative and "expert" based, but quantitative analysis approaches have gained acceptance over the last 30 years (Livint Popa et al., 2020; Maheshwari, 2020). Most laboratories have developed quantitative methods independently and several free and commercial software packages exist for rodent EEG analysis. However, a significant lack of interoperability remains between these tools in handling variability in data type, experimental design, and statistical analysis. This fragmentation forces researchers to expend excessive time and productivity on technical troubleshooting rather than scientific discovery.

NeuRodent was developed to address these challenges by providing a modular, scalable, and interoperable Python-based framework specifically designed for neuroscience researchers working with large-scale rodent cohorts. By unifying analysis pipelines, NeuRodent enables seamless collaboration and analysis comparisons across different laboratories. Key technical features include:

- **Modular Architecture**: A generalized framework for feature calculation allows contributors to easily extend the library of available metrics.

- **Data organization**: A dedicated scheme designed for rodent study analysis, including genotype and experimental day, rather than individual subject sessions.
- **High Interoperability**: Integration with `SpikeInterface` and `MNE-Python` ensures support for a wide array of electrophysiology file formats with syntax frequently used in the field.
- **Scalability**: To address the challenge of analyzing large EEG datasets efficiently, the package integrates dataset caching and uses `Dask` (Dask Development Team, 2016) and `Snakemake` (Mölder et al., 2025) to parallelize computations across high-performance computing clusters.
- **Reproducibility**: Development follows Continuous Integration (CI) practices, and intermediate results are saved to prevent redundant computations following pipeline errors.

State of the Field

The current landscape of electrophysiology and neuroimaging software includes several mature, high-level tools such as `SpikeInterface` (Buccino et al., 2020) and `MNE-Python` (Gramfort et al., 2013) in Python, as well as `EEGLAB` (Delorme & Makeig, 2004), `FieldTrip` (Oostenveld et al., 2011), `Brainstorm` (Tadel et al., 2011), and `Chronux` (Bokil et al., 2010) in MATLAB. These platforms provide powerful building blocks for neurophysiology analysis but are primarily oriented toward human-centric research, high-frequency spike sorting, or local single-session computation. As a result, most rodent EEG analyses remain ad-hoc and designed for local, single-session use rather than maintained and scalable applications. `NeuRodent` provides a unique scholarly contribution and addresses this gap by serving as an orchestration layer that coordinates data loading, analysis, and visualization into reproducible and rodent-specific EEG workflows.

Software Design

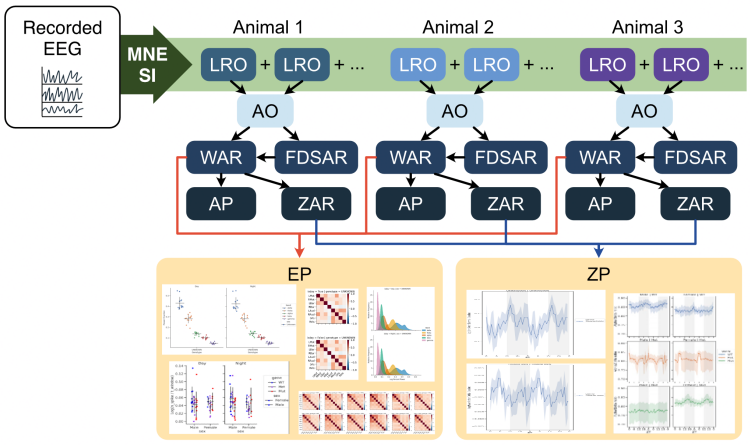


Figure 1: `NeuRodent` package schematic showing data flow through LongRecordingOrganizers (LRO), AnimalOrganizer (AO), WindowAnalysisResult (WAR), FrequencyDomainSpikeAnalysisResult (FDSAR), ZeitgeberAnalysisResult (ZAR), AnimalPlotter (AP), ExperimentPlotter (EP), and ZeitgeberPlotter (ZP).

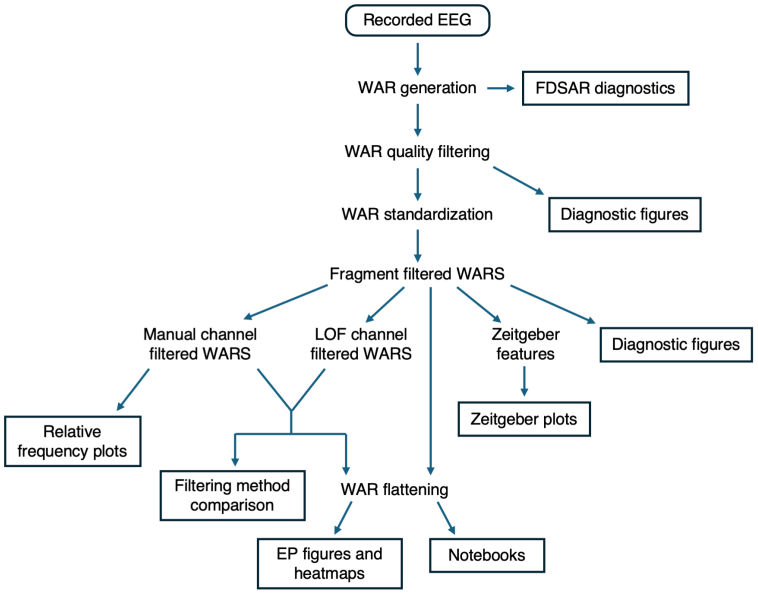


Figure 2: `NeuRodent` `Snakemake` pipeline flowchart showing data flow from raw EEG data to outputs (boxed).

`NeuRodent` is organized as two components: a core Python analysis library (Figure 1), and a `Snakemake` pipeline that orchestrates the library over large datasets (Figure 2). This layered design balances ease of adoption with scalable deployment: per-session or per-animal analyses can use the core library alone and call it from scripts or notebooks, while multi-animal and multi-session processing can use the `Snakemake` pipeline to gain cluster-level orchestration using SLURM. The two components are decoupled, ensuring that changes to the pipeline logic do not force changes on users of the core library, and vice versa.

Rather than implementing its own file readers, `NeuRodent` delegates data loading to `SpikeInterface` and `MNE-Python` (Buccino et al., 2020; Gramfort et al., 2013), which together cover a wide range of electrophysiology formats. These libraries are used for the `spikeinterface.extractors` and `mne.io` file reader functions; users may also supply a custom reader function for novel formats. This deferred approach avoids duplicating format-support effort that is already well maintained by those communities and ensures that `NeuRodent` inherits new format support automatically.

Both libraries are also used as analysis primitives which `NeuRodent` builds on. From `SpikeInterface`, `NeuRodent` uses the `BaseRecording` abstraction, which enables lazy data loading and data streaming to reduce memory pressure, along with recording concatenation and several preprocessing steps for notch filtering, resampling, and scaling; no spike sorting or curation functionality is used. From `MNE-Python`, `NeuRodent` uses `EDF` and `FIF` exporting for file persistence and format conversion, and annotation for spike labeling for users who would like to process or visualize their recordings further with `MNE-Python`. Coherence features are computed with `mne-connectivity` (Binns et al., 2026), while spectral power, correlation, and spike detection are implemented on `SciPy` (Virtanen et al., 2020).

Within the core library, computation is structured around a hierarchy of organizer classes that mirror the stages of a rodent EEG experiment (Figure 1):

- **LongRecordingOrganizer**: one recording session, many channels
- **AnimalOrganizer**: one animal, many recording sessions
- **WindowAnalysisResult**, **FrequencyDomainSpikeAnalysisResult**, **ZeitgeberAnalysisResult**: analysis results of one animal

- **AnimalPlotter**: plots from one animal
- **ExperimentPlotter**, **ZeitgeberPlotter**: plots from many animals

Each of these classes naturally encapsulates a specific scope of rodent EEG/LFP analysis. A nested class hierarchy was chosen over a flat library to make the hierarchy of EEG analysis explicit, with lower level objects composing higher level ones. A practical consequence of this design is that analysis can be parallelized by processing each channel and time window independently. `NeuRodent` uses `Dask` to enable configurable parallel processing of channels and windows, either locally or on a distributed cluster. Adjustable in-memory chunk sizes let users trade throughput for RAM, an important consideration given that EEG recordings can span days or weeks.

`NeuRodent` enables contributors to add new features to compute in windowed analyses by discovering feature computation functions at runtime. This greatly reduces the barrier to contribution for domain scientists who may not be familiar with the broader structure of `NeuRodent`. Artifact rejection is done in a similar fashion, where users can write additional filters and apply them with minimal changes. All computed features are outputted as pandas DataFrames (McKinney, 2010; The pandas development team, 2020) saved in Parquet files, which enables downstream workflows in Excel, R, or other analysis tools to interoperate with `NeuRodent` outputs without needing format conversion.

Data visualization is provided through the Plotter classes. For individual animals, these include channel coherence and correlation matrices, power spectral density histograms, frequency spectrograms, feature time-series, and feature heatmaps. For experiment-wide groups of animals, these include categorical plots, coherence and correlation matrices, Q–Q plots, and 24-hour feature timecourses.

`NeuRodent` is distributed via PyPI and conda-forge for ease of installation and is published on the `Snakemake` workflow catalog. Continuous integration tests cover more than 90% of the core library, and the full `Snakemake` workflow is additionally tested end-to-end on a miniature example dataset, ensuring that changes to the library do not silently break the `Snakemake` pipeline. Clear explanations of installation instructions and tutorials are provided on the `NeuRodent` documentation website. Contributing guidelines and a Makefile-based development setup are provided so that new contributors can get started with a few commands.

Research Impact Statement

Originally designed for EEG analyses for the lead developer, `NeuRodent` is being developed by a team of six code contributors using data across laboratories. The package has been used in a publication characterizing a novel developmental mouse model (Ferrari et al., 2026) and presented at the US Research Software Engineering conference under the name `PyEEG` (Dong et al., 2025). Use beyond the developing laboratories is not yet documented, and the authors intend to run hands-on and recorded departmental workshops to demonstrate the package once the analysis workflow is stable. To lower the barrier to adoption, every tagged release is archived on Zenodo so that a specific analysis version can be cited reproducibly. `NeuRodent` and its accompanying `Snakemake` pipeline will be of great use to neuroscientists performing intracranial LFP analyses on large recorded datasets and distributing analyses across laboratories.

AI Usage Disclosure

No generative AI was used for architectural decisions and scientific interpretations. Claude was used to assist with code development and documentation writing. Code and tests were manually reviewed and edited by human authors, and correctness was validated against large datasets used by our group and collaborating groups.

Acknowledgements

We acknowledge contributions from Jessica Lahr and Ananya Madhira. This project benefited from insights and best practices described in Peter K. G. Williams's *One Good Tutorial*. This work was supported by the Eunice Kennedy Shriver National Institute of Child Health and Human Development (NICHD), National Institutes of Health [P50HD105354]; and by generous donations toward the Marsh Refractory Epilepsy Research Program.

References

Binns, T. S., Li, A., Larson, E., McCloy, D., Rockhill, A., Li, Q., Ruuskanen, S., Kroner, A., Gramfort, A., Marraffini, G. F. G., Shor, J., Marshall, K., Orabe, M., Gerster, M., Köhler, R. M., Steingold, S., & Mert, S. (2026). *MNE-connectivity* (Version 0.8.0). https://doi.org/10.5281/zenodo.10278399

Bokil, H., Andrews, P., Kulkarni, J. E., Mehta, S., & Mitra, P. P. (2010). Chronux: A platform for analyzing neural signals. *Journal of Neuroscience Methods*, *192*(1), 146–151. https://doi.org/10.1016/j.jneumeth.2010.06.020

Buccino, A. P., Hurwitz, C. L., Garcia, S., Magland, J., Siegle, J. H., Hurwitz, R., & Hennig, M. H. (2020). SpikeInterface, a unified framework for spike sorting. *eLife*, *9*, e61834. https://doi.org/10.7554/eLife.61834

Dask Development Team. (2016). *Dask: Library for dynamic task scheduling* [Manual]. http://dask.pydata.org

Delorme, A., & Makeig, S. (2004). EEGLAB: An open source toolbox for analysis of single-trial EEG dynamics including independent component analysis. *Journal of Neuroscience Methods*, *134*(1), 9–21. https://doi.org/10.1016/j.jneumeth.2003.10.009

Dong, J., Oh, Y., Singh, Y., Wei, Y., & Marsh, E. (2025). *PyEEG: Unifying Rodent EEG Analysis Pipelines*. https://doi.org/10.5281/zenodo.17274681

Ferrari, E. K., Dong, J., Duncan-Field, K., Sharma, V., Whipple, S., McCoy, A. J., Marsh, E. D., & Lefebvre, V. (2026). Mice with *Sox5* inactivation in the *Emx1* lineage as a model for the human Lamb-Shaffer neurodevelopmental syndrome. *Brain Research*, *1888*, 150442. https://doi.org/10.1016/j.brainres.2026.150442

Gramfort, A., Luessi, M., Larson, E., Engemann, D. A., Strohmeier, D., Brodbeck, C., Goj, R., Jas, M., Brooks, T., Parkkonen, L., & Hämäläinen, M. (2013). MEG and EEG data analysis with MNE-Python. *Frontiers in Neuroscience*, *7*. https://doi.org/10.3389/fnins.2013.00267

Livint Popa, L., Dragos, H., Pantelemon, C., Verisezan Rosu, O., & Strilciuc, S. (2020). The Role of Quantitative EEG in the Diagnosis of Neuropsychiatric Disorders. *Journal of Medicine and Life*, *13*(1), 8–15. https://doi.org/10.25122/jml-2019-0085

Maheshwari, A. (2020). Rodent EEG: Expanding the Spectrum of Analysis. *Epilepsy Currents*, *20*(3), 149–153. https://doi.org/10.1177/1535759720921377

Marshall, G. F., Gonzalez-Sulser, A., & Abbott, C. M. (2021). Modelling epilepsy in the mouse: Challenges and solutions. *Disease Models & Mechanisms*, *14*(3), dmm047449. https://doi.org/10.1242/dmm.047449

McKinney, W. (2010). Data Structures for Statistical Computing in Python. In S. van der Walt & J. Millman (Eds.), *Proceedings of the 9th Python in Science Conference* (pp. 56–61). https://doi.org/10.25080/Majora-92bf1922-00a

Mölder, F., Jablonski, K. P., Letcher, B., Hall, M. B., van Dyken, P. C., Tomkins-Tinch, C. H., Sochat, V., Forster, J., Vieira, F. G., Meesters, C., Lee, S., Twardziok, S. O., Kanitz, A., VanCampen, J., Malladi, V., Wilm, A., Holtgrewe, M., Rahmann, S., Nahnsen, S., &

Köster, J. (2025). Sustainable data analysis with Snakemake [version 3; peer review: 2 approved]. *F1000Research*, *10*, 33. https://doi.org/10.12688/f1000research.29032.3

Oostenveld, R., Fries, P., Maris, E., & Schoffelen, J.-M. (2011). FieldTrip: Open Source Software for Advanced Analysis of MEG, EEG, and Invasive Electrophysiological Data. *Computational Intelligence and Neuroscience*, *2011*(1), 156869. https://doi.org/10.1155/2011/156869

Tadel, F., Baillet, S., Mosher, J. C., Pantazis, D., & Leahy, R. M. (2011). Brainstorm: A User-Friendly Application for MEG/EEG Analysis. *Computational Intelligence and Neuroscience*, *2011*(1), 879716. https://doi.org/10.1155/2011/879716

The pandas development team. (2020). *Pandas-dev/pandas: Pandas* (latest). Zenodo. https://doi.org/10.5281/zenodo.3509134

Tudor, M., Tudor, L., & Tudor, K. I. (2005). [Hans Berger (1873-1941)–the history of electroencephalography]. *Acta Medica Croatica: Casopis Hravatske Akademije Medicinskih Znanosti*, *59*(4), 307–313. https://www.ncbi.nlm.nih.gov/pubmed/16334737

Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., Burovski, E., Peterson, P., Weckesser, W., Bright, J., van der Walt, S. J., Brett, M., Wilson, J., Millman, K. J., Mayorov, N., Nelson, A. R. J., Jones, E., Kern, R., Larson, E., … SciPy 1.0 Contributors. (2020). SciPy 1.0: Fundamental algorithms for scientific computing in python. *Nature Methods*, *17*, 261–272. https://doi.org/10.1038/s41592-019-0686-2